Project Design

CS6905 – Cloud Information Management Systems

Names	Akinbobola Akin, Alfarizy Alfarizy, Ishimwe Pacis Hanyurwimfura
Student IDs	3784664, 3810253, 3787234
Date	March 12, 2026
Instructor	Dr. Shadi Aljendi

Project Title

Real-Time AIOps Platform for Predictive Incident Detection in Cloud Services

1. Introduction and Summary

This design document turns our CloudWatch AI project proposal into a detailed blueprint that is ready for implementation. In the proposal, we described the core problem that cloud operations teams today still rely on reactive monitoring, meaning they only get alerts after something has already gone wrong and users are already affected. We proposed a machine learning-based approach that predicts incidents before they happen, giving operators enough time to step in and prevent outages.

The purpose of this design phase is to bridge the gap between that high-level idea and what we actually need to build. Specifically, this document covers three things. First, it lays out the full serverless architecture on AWS, showing how simulated cloud service data gets ingested, processed through a feature-engineering pipeline, and fed into a Gradient Boosting classifier for real-time predictions, all within AWS free-tier limits. Second, it defines the DynamoDB data model in detail, explaining how we handle the fact that five different cloud service types (Web API, Database Pool, Message Queue, Authentication Service, and ML Pipeline) each produce logs with completely different structures. Third, it describes how the different components connect together: DynamoDB Streams for triggering the pipeline automatically, API Gateway for serving the dashboard, and EventBridge for scheduling data generation.

In terms of scope, this design covers six DynamoDB tables, nine Lambda functions for simulation, feature engineering, inference, and API handling, a

React-based live dashboard hosted on S3, and the security and scalability measures we need for a reliable demo. The ML model itself, a Gradient Boosting classifier trained on the AIOps KPI Anomaly Detection dataset, which achieved 0.91 precision with about 36 minutes of average lead time, was built and tested during our research phase. This document focuses on how that model fits into the serverless pipeline.

By completing this design, we now have a shared technical reference that lets all three team members work in parallel on the backend infrastructure, ML deployment, and frontend dashboard, while making sure everything lines up in terms of data formats, access patterns, and event flows.

2. System Architecture

The system is fully serverless on AWS, and each stage triggers the next automatically, so there are no polling or manual steps involved.

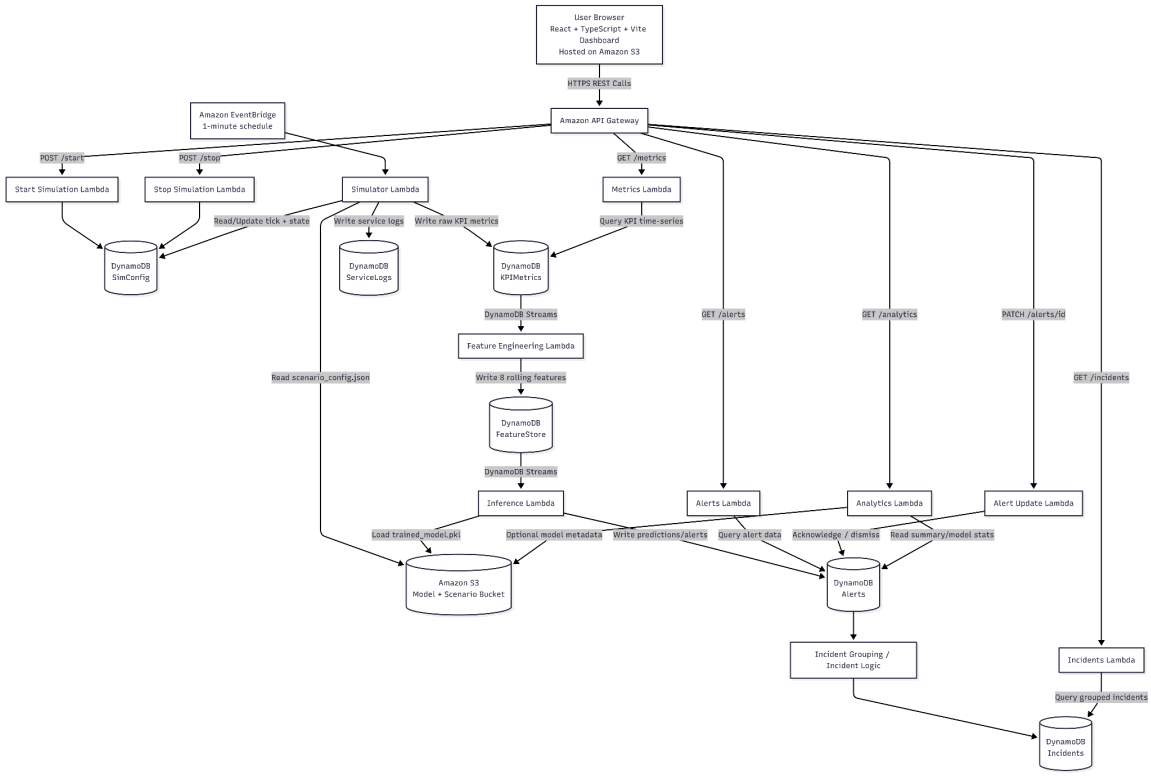
The frontend is a React dashboard hosted on S3. It talks to the backend through API Gateway, which exposes endpoints for controlling the simulation, pulling metrics and alerts, and viewing model analytics.

On the backend side, EventBridge fires the Simulator Lambda once every minute. Each run generates fake KPI metrics and logs for all five cloud services and writes them into two DynamoDB tables, KPIMetrics and ServiceLogs. It also updates a SimConfig table to keep track of where it is in the scenario timeline.

After that, everything is automatic. When new records land in KPIMetrics, DynamoDB Streams kicks off the Feature Engineering Lambda, which computes eight statistical features and writes them to FeatureStore. Another stream on FeatureStore triggers the Inference Lambda, which loads the trained model from S3, runs a prediction, and writes an alert if the score passes the threshold. Related alerts are then grouped into incidents.

On the API side, separate Lambda functions handle each route so that metrics queries, alert lookups, and analytics requests are all isolated and easy to maintain.

Everything stays within the AWS free tier. The simulator runs about 43,200 invocations per month out of the one million limit, and DynamoDB write throughput sits around 5 WCU out of the 25 WCU cap.



3. Data Model Design

We use Amazon DynamoDB as our only database, split into six tables. DynamoDB makes sense here because our five cloud services each produce logs with completely different fields, it handles high-frequency time-series writes out of the box, and DynamoDB Streams lets us trigger pipeline stages automatically without polling.

3.1 Table Overview

The table below shows all six DynamoDB tables, their keys, and what they store.

Table	Partition Key	Sort Key	Purpose
KPIMetrics	service_id	timestamp	Raw metrics from each service. Variable-schema JSON per service type.
ServiceLogs	service_id	timestamp	Event logs with different fields per service. Kept separate for independent access patterns.
FeatureStore	service_id	timestamp	8 computed features per timestep (roll_mean, roll_max, roll_std, roll_slope, first_diff, error_rate, warn_rate, severity_change_flag).
Alerts	alert_id	timestamp	Predictions that crossed the calibrated threshold. Includes confidence score, horizon, and status.
Incidents	incident_id	created_at	Groups of related alerts clustered by time and service. Tracks severity and operator remarks.
SimConfig	config_key	—	Simulation state: tick counter, running flag, active scenario.

3.2 Variable Schema per Service

The main reason we picked DynamoDB over a relational database is that each service type has a completely different set of metrics. Instead of creating a table with lots of nullable columns, we store the metrics as a flexible JSON map inside each KPIMetrics record. Here is what each service produces:

Service Type	Metrics Fields
Web API	response_time_ms, requests_per_second, error_rate, http_5xx_count
Database Pool	active_connections, query_duration_avg_ms, deadlock_count
Message Queue	queue_depth, consumer_lag, dead_letter_queue_size
Auth Service	login_success_rate, auth_latency_ms, mfa_challenge_rate
ML Pipeline	inference_latency_ms, model_accuracy, data_drift_score

3.3 Indexing and Partitioning

All data tables use service_id as the partition key and timestamp as the sort key. This means all records for a given service are stored together, making time-range queries fast (for example, "get the last 30 minutes of web-api-001"). With 5 services writing 1 record per minute each, the write workload spreads evenly and stays well within the free-tier limit of 25 WCU.

We add two Global Secondary Indexes (GSIs) on the Alerts table: one on service_id + timestamp (to pull all alerts for a specific service) and one on status + timestamp (to show active alerts on the dashboard). A GSI on service_type + timestamp is also added to KPIMetrics for cross-service dashboard queries.

All data tables have TTL enabled. Records older than 24 hours are automatically deleted, so we stay within the 25 GB free-tier storage limit without needing any manual cleanup.

3.4 Data Flow Between Tables

DynamoDB Streams acts as the glue between pipeline stages. Each stage automatically triggers the next, so there is no polling and no manual orchestration:

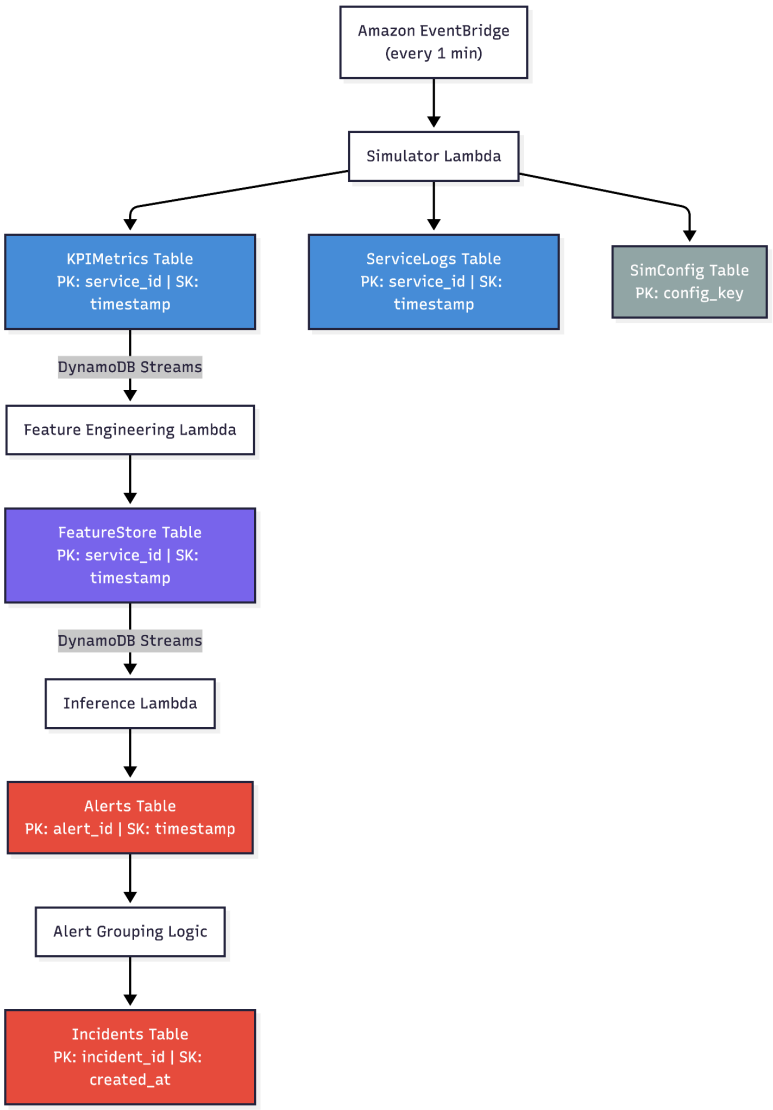
Step	Trigger	What Happens	Writes To
1	EventBridge (every 1 min)	Simulator generates fake metrics and logs	KPIMetrics, ServiceLogs
2	Streams on KPIMetrics	Feature Engineering computes 8 features	FeatureStore
3	Streams on FeatureStore	Inference Lambda runs ML model	Alerts
4	Alert grouping logic	Clusters related alerts together	Incidents

3.5 Key Design Decisions

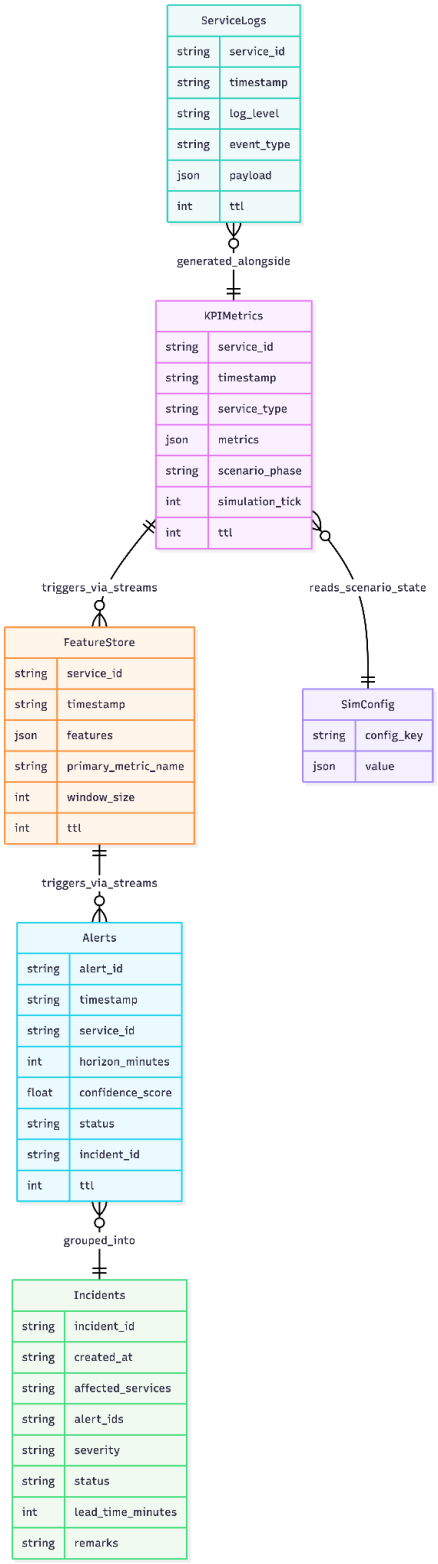
Decision	Why
Separate KPIMetrics and ServiceLogs	KPIMetrics feeds the ML pipeline; ServiceLogs are just for dashboard display. Different access patterns and retention needs.
Flexible Map attributes for metrics	Each service type defines its own fields in JSON. No nullable columns needed.
Composite keys (service_id + timestamp)	Efficient per-service time-range queries and even partition distribution.
DynamoDB Streams as pipeline glue	No polling. Each stage triggers the next automatically, reducing latency and Lambda invocations.
TTL on all data tables	Automatic cleanup to stay within the 25 GB free-tier limit.

3.6 Diagrams

Data Flow Diagram:



Entity Relationship Diagram:



4. Security and Scalability Considerations

4.1 Security

Our system follows cloud security best practices to protect system data and prevent unauthorized access.

First, all communication between the frontend dashboard and backend services is performed over HTTPS, ensuring encryption of data in transit. This prevents interception of KPI metrics, prediction results, and system commands.

Second, AWS Identity and Access Management (IAM) is used to control permissions between services. Each Lambda function is assigned a role with only the permissions it needs (principle of least privilege). For example, the Simulator Lambda can only write to KPIMetrics and ServiceLogs, the Inference Lambda can only read from FeatureStore and write to Alerts, and only the Inference Lambda has permission to read the trained model from S3.

Third, sensitive data stored in Amazon DynamoDB and Amazon S3 is protected using AWS-managed encryption. This ensures that stored metrics, logs, and ML model artifacts remain protected even if storage resources are accessed improperly and also TTL on DynamoDB tables that auto-deletes old records.

Finally, API Gateway acts as the secure entry point to the backend. It provides request validation and rate limiting to protect the system from malformed requests or excessive traffic.

Since the platform only processes synthetic data generated by our simulator, there is no real user data or personally identifiable information involved. In a production setting, real service logs would need to be anonymized before ingestion to comply with data privacy requirements.

4.2 Scalability and Performance

The system is designed using a serverless architecture, which allows it to scale automatically with workload demand.

AWS Lambda functions automatically scale based on incoming events. If multiple services generate KPI data at the same time, we can launch multiple Lambda instances to process the workload concurrently without manual infrastructure management.

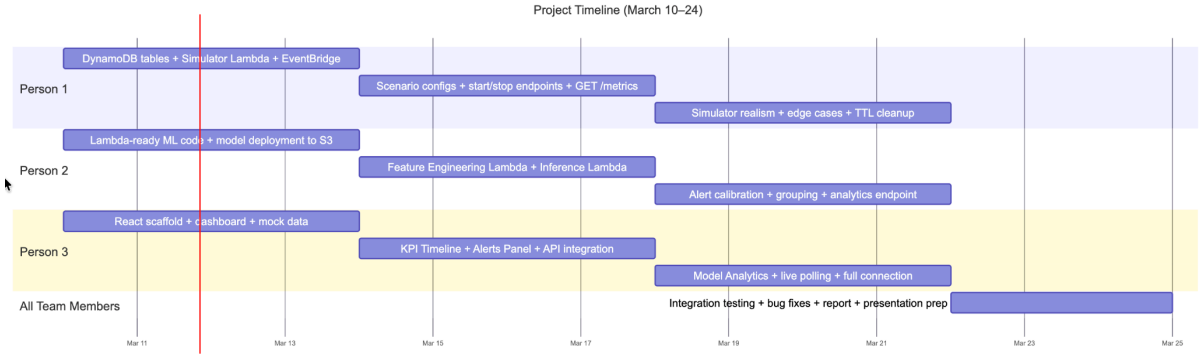
The platform also uses event-driven processing through DynamoDB Streams. When new KPI data is inserted, streams automatically trigger the feature engineering stage, which then triggers the inference stage. This asynchronous pipeline allows each stage to scale independently and avoids bottlenecks.

We are also using Amazon DynamoDB, which provides horizontal scalability and low-latency access for time-series metrics and prediction results. It can automatically handle increasing data volume as the number of monitored services grows.

And finally, the frontend dashboard is hosted as a static web application on Amazon S3, allowing it to scale easily and serve multiple users without affecting backend processing performance.

5. Timeline and Risk Assessment

5.1 Timeline



5.2 Risk Assessment

One potential risk is prediction inaccuracy from the machine learning model. If the model produces too many false alerts, users may lose trust in the system. This risk will be mitigated by tuning prediction thresholds and evaluating model precision during testing.

Another risk is pipeline latency. Since the system processes events through multiple Lambda stages, delays could occur before predictions are generated. This will be addressed by optimizing Lambda execution time and minimizing unnecessary computations.

A third risk involves AWS service limits, such as Lambda concurrency limits or API Gateway request throttling. Monitoring and configuration adjustments will be used to ensure the system can handle increased workloads.

Finally, data consistency delays in DynamoDB could cause temporary mismatches between stored metrics and displayed dashboard values. The dashboard will tolerate small delays and refresh data periodically to ensure updated information is displayed.

References

Dang, Y., Lin, Q., & Huang, P. (2019). AIOps: Real-world challenges and research innovations. In *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)* (pp. 4–5). IEEE. https://doi.org/10.1109/ICSE-Companion.2019.00023

Li, Z., Zhao, N., Zhang, S., Sun, Y., Chen, P., Wen, X., Ma, M., & Pei, D. (2022). Constructing large-scale real-world benchmark datasets for AIOps. *arXiv preprint arXiv:2208.03938*. https://arxiv.org/abs/2208.03938

Zhao, N., Chen, J., Wang, Z., Peng, X., Wang, G., Wu, Y., Zhou, F., Feng, Z., Nie, X., Zhang, W., Sui, K., & Pei, D. (2020). Real-time incident prediction for online service systems. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)* (pp. 315–326). ACM. https://doi.org/10.1145/3368089.3409672

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Amazon Web Services. (2024). *Amazon DynamoDB developer guide*. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/

Amazon Web Services. (2024). *AWS Lambda developer guide*. https://docs.aws.amazon.com/lambda/latest/dg/

Amazon Web Services. (2024). *Amazon API Gateway developer guide*. https://docs.aws.amazon.com/apigateway/latest/developerguide/

Amazon Web Services. (2024). *Amazon DynamoDB Streams developer guide*. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html

Amazon Web Services. (2024). *Amazon EventBridge user guide*. https://docs.aws.amazon.com/eventbridge/latest/userguide/

Meta Platforms. (2024). *React: A JavaScript library for building user interfaces*. https://react.dev/

Amazon Web Services. (2024). *AWS Well-Architected Framework: Serverless Applications Lens*. https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/welcome.html

Notaro, P., Cardoso, J., & Gerndt, M. (2021). A systematic mapping study in AIOps. In *Proceedings of the International Conference on Service-Oriented Computing (ICSOC 2020)* (pp. 110–123). Springer. https://doi.org/10.1007/978-3-030-76352-7_15

Amazon Web Services. (2024). *AWS Identity and Access Management (IAM) user guide*. https://docs.aws.amazon.com/IAM/latest/UserGuide/

Amazon Web Services. (2024). *Amazon Simple Storage Service (S3) user guide*. https://docs.aws.amazon.com/AmazonS3/latest/userguide/